

SNOWFLAKE

Complete Reference Guide

For 5+ Years Experience

Architecture · Loading · Performance · Security · Pipelines · Advanced Features

Theory + Code + Interview Questions + Scenario-Based Questions

Table of Contents

Chapter 1 — Snowflake Architecture & Foundations

- Three-layer architecture
- Virtual warehouses
- Caching layers
- Micro-partitioning

Chapter 2 — Tables, Storage & Time Travel

- Table types
- Time Travel & Fail-safe
- Zero-copy cloning
- Sequences

Chapter 3 — Data Loading & Ingestion

- File formats
- Stages & types
- COPY INTO
- AWS S3 loading
- Snowpipe
- INFER_SCHEMA

Chapter 4 — SQL, Performance & Semi-Structured Data

- Query optimization
- Clustering keys
- Window functions
- VARIANT & JSON
- LATERAL FLATTEN

Chapter 5 — Views & Materialized Views

- Standard views
- Secure views
- Materialized views
- Recursive views

Chapter 6 — Streams, Tasks & Pipelines

- Stream types
- Task types & DAGs
- SCD Type 2
- Task monitoring & SNS

Chapter 7 — Security & Governance

- RBAC & role hierarchy
- Dynamic data masking
- Row access policies
- Data sharing
- Network policies

Chapter 8 — Advanced Features

- Stored procedures
- UDFs & UDTFs
- Snowpark
- Dynamic tables
- Resource monitors

Chapter 9 — Interview Questions

- 50 technical questions
- 40 scenario-based questions
- Architecture deep-dives
- Debugging scenarios

CHAPTER 1

Snowflake Architecture & Foundations

Understanding Snowflake's unique cloud-native architecture is the foundation for everything else. The separation of storage and compute is what makes Snowflake fundamentally different from all traditional databases.

1.1 The Three-Layer Architecture

Snowflake is built on three fully independent layers. Each layer scales separately, fails independently, and is managed by Snowflake automatically. This design is the source of most of Snowflake's key advantages.

Layer	Name	What It Does	Who Manages It
Layer 1	Cloud Services	Authentication, query optimization, metadata, access control, transaction management	Snowflake (always running, billed up to 10% of compute free)
Layer 2	Query Processing	Virtual warehouses — compute clusters that execute queries	You (create, size, suspend, resume)
Layer 3	Storage	Compressed columnar micro-partition files in S3/Azure/GCS	Snowflake (you never touch cloud storage directly)

The critical insight: Layer 3 (storage) and Layer 2 (compute) are completely decoupled. Multiple virtual warehouses can read the same data simultaneously with zero contention. You pay for compute only when it is running.

EXAM Exam: The separation of storage and compute means you can have 10 warehouses all querying the same table at the same time.

1.2 Virtual Warehouses

A virtual warehouse is a cluster of compute nodes (CPU + RAM + local SSD) that Snowflake provisions on demand. It reads data from the storage layer, executes queries, and returns results. When idle it can be suspended — billing stops immediately.

Warehouse Sizes & Credits

Size	Nodes	Credits/Hour	Best For
X-Small	1	1	Development, testing, light workloads
Small	2	2	Staging, small ETL
Medium	4	4	Standard analytics, COPY INTO
Large	8	8	Heavy ETL, large aggregations
X-Large	16	16	Very large joins, ML feature prep
2X-Large	32	32	Massive datasets, parallel processing

4X-Large	64	64	Extreme scale analytics
6X-Large	128	128	Largest ML / data science workloads

NOTE Each step up doubles nodes AND doubles cost. Scale UP for slow complex queries. Scale OUT (multi-cluster) for ma

Virtual Warehouse — Creation & Management

```
-- Create warehouse with all key properties
CREATE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE      = 'MEDIUM'
  WAREHOUSE_TYPE      = 'STANDARD'      -- or SNOWPARK-OPTIMIZED (16x memory for ML)
  AUTO_SUSPEND        = 60              -- suspend after 60s idle (saves cost)
  AUTO_RESUME         = TRUE            -- wake automatically on next query
  MIN_CLUSTER_COUNT   = 1              -- multi-cluster: minimum clusters
  MAX_CLUSTER_COUNT   = 4              -- multi-cluster: scale out to 4 (Enterprise)
  SCALING_POLICY       = 'STANDARD'     -- STANDARD=fast scale, ECONOMY=save cost
  COMMENT             = 'Analytics team';

-- Resize on the fly (running queries complete first)
ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'LARGE';

-- Suspend manually (billing stops immediately)
ALTER WAREHOUSE analytics_wh SUSPEND;

-- Resume manually
ALTER WAREHOUSE analytics_wh RESUME;

-- Monitor warehouse usage
SHOW WAREHOUSES;
SELECT WAREHOUSE_NAME, SUM(CREDITS_USED) AS total_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATE_TRUNC('month', CURRENT_TIMESTAMP())
GROUP BY 1 ORDER BY 2 DESC;
```

Warehouse Types

Type	Memory	Use Case
Standard	Normal	SQL queries, ETL, analytics
Snowpark-Optimized	16x per node	Python/ML workloads, large DataFrames
Serverless Compute	Auto-managed	Snowpipe, serverless tasks, search optimization

EXAM Billing: per-second with a 60-second MINIMUM on every resume. Suspending a warehouse clears the local disk cache

1.3 The Three Caching Layers

Snowflake has three independent caches that serve queries at different speeds. Understanding them is essential for performance optimization.

Cache	Location	Duration	Bypassed By	Requires Warehouse?
Result Cache	Cloud Services layer	24 hours (resets on hit)	Non-deterministic functions (CURRENT_TIMESTAMP, RANDOM)	No — free
Local Disk Cache	Warehouse SSD	Until warehouse suspends	Warehouse suspend/resume	Yes — already running
Metadata Cache	Cloud Services layer	Always warm	Nothing — always available	No — free

Caching — Examples

```
-- Test result cache
SELECT COUNT(*) FROM orders WHERE region = 'US';
-- Run again immediately → 0ms execution (result cache hit)

-- Bypass result cache (non-deterministic function)
SELECT COUNT(*), CURRENT_TIMESTAMP() FROM orders; -- always miss

-- Disable result cache for testing
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
ALTER SESSION SET USE_CACHED_RESULT = TRUE; -- re-enable

-- Metadata cache: instant for min/max, count
SELECT MIN(order_date), MAX(order_date) FROM orders; -- served from metadata
SELECT COUNT(*) FROM orders; -- served from metadata
```

1.4 Micro-Partitioning

Snowflake automatically divides every table into immutable compressed columnar files called micro-partitions, each containing 50–500 MB of uncompressed data. Each partition stores metadata: min value, max value, null count, and distinct count per column. The query optimizer uses this metadata to skip entire partitions — this is called partition pruning.

Micro-Partitioning & Clustering

```
-- Check clustering health of a table
SELECT SYSTEM$CLUSTERING_INFORMATION('orders', '(order_date)');
-- Returns: average_depth (lower = better), overlap_depth, total_partition_count

-- Check partition pruning in query plan
EXPLAIN SELECT * FROM orders WHERE order_date = '2024-03-15';
-- Look for: "partitionsAssigned" vs "partitionsTotal" — lower ratio = better

-- Add clustering key on heavily filtered column
ALTER TABLE orders CLUSTER BY (order_date);

-- Composite clustering (for multi-column filters)
ALTER TABLE orders CLUSTER BY (order_date, region);

-- Check if reclustering is needed
SELECT SYSTEM$CLUSTERING_INFORMATION('orders');
-- average_depth > 3 = consider reclustering
```

```
-- Drop clustering key
```

EXAM Clustering keys are worth adding ONLY on tables >1 TB with consistent filter patterns on high-cardinality columns. No

CHAPTER 2

Tables, Storage & Time Travel

Snowflake supports five table types with different durability guarantees, storage costs, and use cases. Time Travel and Fail-safe provide powerful data recovery capabilities.

2.1 Table Types

Table Type	Time Travel	Fail-safe	Scope	Storage Cost	Best For
Permanent	Up to 90 days	7 days	Account	Highest	Production data
Transient	Up to 1 day	None	Account	Lower	Staging, intermediate ETL
Temporary	Up to 1 day	None	Session only	Minimal	Within-session temp work
External	None	None	Account	None (files in cloud)	Query files without loading
Dynamic	Up to 90 days	7 days	Account	Standard	Declarative incremental pipelines

All Table Types — Creation Examples

```
-- PERMANENT TABLE (default) -- full Time Travel + Fail-safe
CREATE TABLE orders (
  order_id    NUMBER AUTOINCREMENT PRIMARY KEY,
  customer_id NUMBER,
  amount      FLOAT,
  region      VARCHAR(50),
  status      VARCHAR(20) DEFAULT 'pending',
  order_date  DATE,
  created_at  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- TRANSIENT TABLE -- no Fail-safe, lower storage cost
-- Use for staging tables that can be recreated if lost
CREATE TRANSIENT TABLE stg_orders_raw (
  raw_json    VARIANT,
  loaded_at   TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- TEMPORARY TABLE -- session-scoped, invisible to other sessions
CREATE TEMPORARY TABLE tmp_order_calc AS
SELECT order_id, amount * 1.1 AS adjusted_amount FROM orders;

-- EXTERNAL TABLE -- query S3/GCS/Azure files as a table
CREATE EXTERNAL TABLE ext_orders (
  order_id    NUMBER    AS (VALUE:c1::NUMBER),
  amount      FLOAT     AS (VALUE:c2::FLOAT),
  region      VARCHAR   AS (VALUE:c3::VARCHAR)
)
WITH LOCATION      = @my_s3_stage
FILE_FORMAT        = (TYPE='CSV' SKIP_HEADER=1)
FORMAT_NAME        = 'CSV'

```

2.2 Time Travel

Time Travel lets you query, clone, or restore data from any past point within the retention period. Standard edition: up to 1 day. Enterprise edition: up to 90 days. Retention is configurable per object.

Time Travel — Full Reference

```
-- Query data as it was 1 hour ago
SELECT * FROM orders AT (OFFSET => -3600);

-- Query at a specific timestamp
SELECT * FROM orders
AT (TIMESTAMP => '2024-03-15 09:00:00':TIMESTAMP_LTZ);

-- Query just BEFORE a specific query ran (use Query History to find ID)
SELECT * FROM orders BEFORE (STATEMENT => '01b2c3d4-abcd-1234-efgh-567890abcdef');

-- Recover accidentally deleted rows
INSERT INTO orders
SELECT * FROM orders BEFORE (STATEMENT => '01b2c3d4...')
WHERE order_id NOT IN (SELECT order_id FROM orders);

-- Restore entire table from before a bad DELETE
CREATE TABLE orders_restored AS
SELECT * FROM orders BEFORE (STATEMENT => '01b2c3d4...');

-- Recover a dropped table (must be within retention period)
DROP TABLE orders;
UNDROP TABLE orders;           -- restores it!

-- Recover dropped schema or database
UNDROP SCHEMA my_schema;
UNDROP DATABASE my_database;

-- Set retention per object
ALTER TABLE orders SET DATA_RETENTION_TIME_IN_DAYS = 30;
ALTER TABLE stg_raw SET DATA_RETENTION_TIME_IN_DAYS = 1;  -- transient-like

-- Check storage consumed by Time Travel
SELECT TABLE_NAME,
       ACTIVE_BYTES/1e9      AS active_gb,
       TIME_TRAVEL_BYTES/1e9 AS time_travel_gb,
       FAILSAFE_BYTES/1e9    AS failsafe_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.TABLE_STORAGE_METRICS
ORDER BY (TIME_TRAVEL_BYTES + FAILSAFE_BYTES) DESC LIMIT 10;
```

EXAM Fail-safe: 7 days AFTER Time Travel expires. NOT accessible via SQL — only Snowflake Support can recover. Settings

2.3 Zero-Copy Cloning

Zero-copy cloning creates an instant copy of a table, schema, or database by duplicating only the metadata pointer. The source and clone share the same micro-partitions until one is modified. Only changed partitions are then physically duplicated.

Zero-Copy Cloning

TIP Zero-copy clone + Time Travel is the fastest disaster recovery pattern. Always clone before a risky migration, not after s

CHAPTER 3

Data Loading & Ingestion

Snowflake provides multiple mechanisms for loading data: bulk loading via COPY INTO, continuous loading via Snowpipe, and direct querying of external files via external tables.

3.1 File Formats

File Formats — All Types

```
-- CSV (most common for batch loads)
CREATE FILE FORMAT csv_fmt
  TYPE = 'CSV'
  FIELD_DELIMITER = ','
  RECORD_DELIMITER = '
'
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '\N', '')
  EMPTY_FIELD_AS_NULL = TRUE
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  ESCAPE = '\\
'
  COMPRESSION = AUTO; -- auto-detect gzip, bzip2, zstd

-- JSON (for nested/semi-structured data)
CREATE FILE FORMAT json_fmt
  TYPE = 'JSON'
  STRIP_OUTER_ARRAY = TRUE -- if file is [{...},{...}] wrap in array
  STRIP_NULL_VALUES = FALSE
  COMPRESSION = AUTO;

-- Parquet (columnar – fastest, schema embedded in file)
CREATE FILE FORMAT parquet_fmt
  TYPE = 'PARQUET'
  SNAPPY_COMPRESSION = TRUE;

-- ORC (columnar – Hive/Hadoop ecosystem)
CREATE FILE FORMAT orc_fmt
  TYPE = 'ORC';

-- Avro (row-based, schema embedded, Kafka-native)
CREATE FILE FORMAT avro_fmt
  TYPE = 'AVRO';

-- XML
CREATE FILE FORMAT xml_fmt
  TYPE = 'XML'
  STRIP_OUTER_ELEMENT = TRUE;

-- List available file formats
SHOW FILE FORMATS;
```

3.2 Stages & Types

A stage is a named location (internal or external) where data files are stored before loading. Every file must pass through a stage — you cannot load directly from your laptop to a table.

Stage Type	Reference Syntax	Created By	Scope	Use Case
User stage	@~	Auto (per user)	Per user	Personal quick uploads
Table stage	@%table_name	Auto (per table)	Per table	Load only that specific table
Named internal	@stage_name	CREATE STAGE	Account	Shared production stages
External (S3)	@stage_name	CREATE STAGE + integration	Account	Files already in your cloud storage

Stages — All Types

```
-- USER STAGE — automatic, reference with @~
PUT file:///Users/me/data/orders.csv @~;
LIST @~;

-- TABLE STAGE — automatic per table, reference with +%table_name
PUT file:///data/orders.csv +%orders;
COPY INTO orders FROM +%orders FILE_FORMAT=(FORMAT_NAME='csv_fmt');

-- NAMED INTERNAL STAGE — explicitly created, reusable
CREATE STAGE my_stage
  FILE_FORMAT = (FORMAT_NAME = 'csv_fmt')
  COMMENT     = 'Production ETL stage';

-- EXTERNAL STAGE — AWS S3 with storage integration (recommended)
CREATE STORAGE INTEGRATION s3_int
  TYPE                = EXTERNAL_STAGE
  STORAGE_PROVIDER    = 'S3'
  ENABLED              = TRUE
  STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/data/');

-- Get IAM ARN for trust policy setup in AWS
DESC INTEGRATION s3_int;

CREATE STAGE s3_stage
  URL                = 's3://my-bucket/data/'
  STORAGE_INTEGRATION = s3_int
  FILE_FORMAT        = (FORMAT_NAME = 'csv_fmt');

-- EXTERNAL STAGE — Azure Blob
CREATE STAGE azure_stage
  URL                = 'azure://myaccount.blob.core.windows.net/container/'
  STORAGE_INTEGRATION = azure_int;

-- Stage operations
LIST @my_stage;                -- list files
REMOVE @my_stage/orders_old.csv; -- delete file
GET @my_stage/file.csv file:///tmp; -- download (SnowSQL only)
```

3.3 COPY INTO — Bulk Loading & Unloading

COPY INTO — Full Reference

[illegible]

3.4 Continuous Loading with Snowpipe

Snowpipe is an event-driven continuous loading service. When files land in a stage, Snowpipe automatically runs the associated COPY INTO definition. It uses serverless compute — billed per file rather than per warehouse hour.

Snowpipe — AUTO_INGEST Setup

```
-- Create pipe with AUTO_INGEST (S3 event notifications)
CREATE PIPE orders_pipe
  AUTO_INGEST = TRUE
  COMMENT     = 'Auto-load orders from S3'
AS
  COPY INTO orders
  FROM        @s3_stage
  FILE_FORMAT = (FORMAT_NAME = 'csv_fmt');

-- Step: get the SQS ARN for S3 event notification setup
SHOW PIPES;
-- Note: notification_channel column = SQS queue ARN
-- In AWS: S3 bucket → Properties → Event Notifications → ObjectCreated → SQS → paste ARN

-- Monitor pipe status
SELECT SYSTEM$PIPE_STATUS('orders_pipe');
-- Check: executionState (RUNNING/PAUSED), pendingFileCount, lastIngestedTimestamp

-- Catch up on missed files (run after outage or initial setup)
ALTER PIPE orders_pipe REFRESH;
ALTER PIPE orders_pipe REFRESH
  PREFIX          = 'orders/2024/03/'
  MODIFIED_AFTER = '2024-03-01T00:00:00';

-- Pause and resume
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = TRUE;
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;

-- Pipe load history
SELECT * FROM TABLE(INFORMATION_SCHEMA.PIPES_USAGE_HISTORY(
  DATE_RANGE_START => DATEADD('day', -1, CURRENT_TIMESTAMP()),
  PIPE_NAME         => 'ORDERS_PIPE'
));
```

3.5 INFER_SCHEMA / AUTO_DETECT

INFER_SCHEMA & AUTO_DETECT

```
-- Infer schema from Parquet/ORC/Avro/JSON/CSV files
SELECT * FROM TABLE(
  INFER_SCHEMA(
    LOCATION      => '@my_stage/orders.parquet',
    FILE_FORMAT   => 'parquet_fmt'
  )
);

-- Returns: COLUMN_NAME, TYPE, NULLABLE, EXPRESSION, FILENAMES

-- Auto-create table from inferred schema
```

CHAPTER 4

SQL, Performance & Semi-Structured Data

Snowflake's columnar storage and partition pruning enable dramatic performance gains when queries are written to leverage the architecture. Semi-structured VARIANT columns allow native JSON/Parquet/Avro storage without schema enforcement.

4.1 Query Performance Optimization

Performance optimization in Snowflake follows a hierarchy: result cache → local disk cache → partition pruning → warehouse sizing. Most performance issues are partition pruning problems that clustering keys can solve.

Query Optimization Techniques

```
-- Check query execution in Query Profile (UI):
-- Partitions scanned / total → high ratio = add clustering key
-- Bytes spilled to local storage → warehouse too small, size up
-- Bytes scanned from remote storage → cold warehouse, keep warm

-- Check partition pruning before running
EXPLAIN SELECT SUM(amount) FROM orders WHERE order_date = '2024-03-15';

-- Select only needed columns (columnar storage – fewer columns = less I/O)
-- BAD:
SELECT * FROM orders WHERE region = 'US';
-- GOOD:
SELECT order_id, amount, order_date FROM orders WHERE region = 'US';

-- Filter early in CTEs before joins
WITH us_orders AS (
  SELECT order_id, customer_id, amount
  FROM orders
  WHERE region = 'US'           -- prune partitions FIRST
    AND order_date >= '2024-01-01'
)
SELECT u.order_id, c.name, u.amount
FROM us_orders u
JOIN customers c ON u.customer_id = c.customer_id;

-- QUALIFY – Snowflake-specific, filters on window functions without subquery
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1;
-- Gets most recent order per customer – cleaner than subquery approach

-- Avoid implicit type casts on join keys (prevents partition pruning)
-- BAD: orders.customer_id is NUMBER, customers.id is VARCHAR
-- GOOD: ensure join key types match:
ALTER TABLE customers MODIFY COLUMN id TYPE NUMBER;
```

4.2 Window Functions

Window Functions — Complete Reference

4.3 Semi-Structured Data — VARIANT, OBJECT, ARRAY

Snowflake stores JSON, Avro, Parquet, ORC, and XML natively in VARIANT columns. No schema is required at load time. Queries use colon notation to traverse JSON paths.

VARIANT & Semi-Structured Data

```
-- Create table with VARIANT column
CREATE TABLE events (
  event_id  VARCHAR DEFAULT UUID_STRING(),
  loaded_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  raw_data  VARIANT
);

-- Insert JSON
INSERT INTO events (raw_data)
SELECT PARSE_JSON('{
  "user_id": 42,
  "action": "purchase",
  "amount": 149.99,
  "items": [
    {"product": "shoes", "qty": 1, "price": 89.99},
    {"product": "socks", "qty": 3, "price": 19.99}
  ],
  "meta": {"device": "mobile", "country": "US", "os": "iOS"}
}');

-- Query JSON with colon notation + explicit cast
SELECT
  raw_data:user_id::NUMBER      AS user_id,
  raw_data:action::VARCHAR      AS action,
  raw_data:amount::FLOAT        AS amount,
  raw_data:meta.device::VARCHAR AS device,      -- dot notation for nested
  raw_data:meta['country']::VARCHAR AS country  -- bracket notation
FROM events;

-- Array element by index (0-based)
SELECT raw_data:items[0]:product::VARCHAR AS first_item FROM events;

-- LATERAL FLATTEN – explode array into rows (most important semi-struct function)
SELECT
  e.event_id,
  f.index      AS item_index,
  f.value:product::VARCHAR AS product,
  f.value:qty::NUMBER      AS quantity,
  f.value:price::FLOAT     AS price
FROM events e,
LATERAL FLATTEN(INPUT => e.raw_data:items) f;
-- f.value = the array element
-- f.index = 0-based position
-- f.key   = object key (for objects)
-- f.path  = full path

-- FLATTEN object keys into rows
SELECT f.key, f.value::VARCHAR
FROM events,
LATERAL FLATTEN(INPUT => raw_data:meta) f;
```

CHAPTER 5

Views & Materialized Views

Views in Snowflake range from simple virtual queries to pre-computed materialized results. Secure views hide their definitions and are required for data sharing.

5.1 View Types

View Type	Stores Data?	Hides Definition?	Auto-Refresh?	Edition
Standard	No	No	N/A	All
Secure	No	Yes	N/A	All
Materialized	Yes	Optional	Yes (background)	Enterprise+
Recursive	No	No	N/A	All

Views — All Types

```
-- STANDARD VIEW – reruns SQL on every query
CREATE VIEW us_customers_v AS
  SELECT customer_id, name, email, region
  FROM customers
  WHERE region = 'US';

-- SECURE VIEW – hides SQL definition from non-owners
-- Required for Snowflake Data Sharing
CREATE SECURE VIEW us_customers_secure_v AS
  SELECT customer_id, name, email, region
  FROM customers
  WHERE region = 'US';

-- Check if a view is secure
SHOW VIEWS LIKE 'us_customers%';
-- is_secure column = TRUE for secure views

-- MATERIALIZED VIEW – pre-computed, stored physically, auto-refreshed
CREATE MATERIALIZED VIEW daily_sales_mv AS
  SELECT
    DATE(order_date) AS sale_day,
    region,
    SUM(amount)      AS total_sales,
    COUNT(*)         AS order_count
  FROM orders
  GROUP BY 1, 2;
-- Limitations: single table only, no joins, no UDFs, no non-deterministic functions
-- Enterprise edition required. Billed for auto-refresh compute.

SHOW MATERIALIZED VIEWS; -- check refresh status

-- RECURSIVE VIEW – for hierarchical data (org charts, bill of materials)
CREATE RECURSIVE VIEW org_v (emp_id, name, mgr_id, level, path) AS
  -- Anchor: top-level (no manager)
  SELECT emp_id, name, mgr_id, 1, name
  FROM employees WHERE mgr_id IS NULL
  -- Recursive: employees under each manager
```

EXAM Secure views prevent definition exposure even to users with USAGE privilege. Materialized views auto-refresh in the

CHAPTER 6

Streams, Tasks & Pipelines

Streams track data changes (CDC) and tasks schedule SQL execution. Together they form Snowflake's native pipeline engine — no external orchestrator needed for many use cases.

6.1 Streams & Types

Stream Type	Tracks	ON Clause	Best For
Standard	INSERT + UPDATE + DELETE	ON TABLE	Full CDC pipelines, SCD2
Append-only	INSERT only	ON TABLE ... APPEND_ONLY=TRUE	Log tables, event streams
Insert-only	INSERT only	ON EXTERNAL TABLE ... INSERT_ONLY=TRUE	External table ingestion

Streams — All Types & Consumption

```
-- STANDARD STREAM – tracks all DML changes
CREATE STREAM orders_stream ON TABLE orders;

-- APPEND-ONLY STREAM – inserts only, more efficient
CREATE STREAM orders_append ON TABLE orders APPEND_ONLY = TRUE;

-- INSERT-ONLY STREAM – for external tables
CREATE STREAM ext_stream ON EXTERNAL TABLE ext_orders INSERT_ONLY = TRUE;

-- Stream metadata columns (auto-added by Snowflake)
SELECT
  order_id,
  amount,
  METADATA$ACTION,          -- 'INSERT' or 'DELETE'
  METADATA$ISUPDATE,       -- TRUE if part of an UPDATE (appears as DELETE+INSERT pair)
  METADATA$ROW_ID          -- Snowflake's internal unique row identifier
FROM orders_stream;

-- Check if stream has unconsumed data
SELECT SYSTEM$STREAM_HAS_DATA('orders_stream'); -- TRUE / FALSE

-- Consume stream with MERGE (offset advances only on successful COMMIT)
MERGE INTO orders_target t
USING (
  SELECT * FROM orders_stream
) s ON t.order_id = s.order_id
WHEN MATCHED AND s.METADATA$ACTION = 'DELETE' THEN DELETE
WHEN MATCHED AND s.METADATA$ISUPDATE = TRUE THEN UPDATE SET
  t.amount = s.amount, t.status = s.status, t.updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED AND s.METADATA$ACTION = 'INSERT' THEN INSERT
  (order_id, amount, status) VALUES (s.order_id, s.amount, s.status);
-- After successful COMMIT: stream offset advances, consumed rows disappear

-- View stream properties
SHOW STREAMS;
```

NOTE Reading a stream with plain SELECT does NOT advance the offset. The offset advances ONLY when you consume t

6.2 Tasks & Types

Task Type	Has Schedule?	Compute	Triggered By	Best For
Standalone	Yes	Warehouse or serverless	Schedule / EXECUTE TASK	Single-step scheduled jobs
Root (DAG)	Yes	Warehouse or serverless	Schedule	Start of a pipeline DAG
Child (DAG)	No	Warehouse or serverless	Parent task success	Steps after root in pipeline
Serverless	Yes	Auto-managed	Schedule / EXECUTE TASK	Variable-volume, cost-efficient

Tasks — All Types & DAG Pattern

```
-- STANDALONE TASK with stream guard
CREATE TASK process_orders_task
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream') -- skip if no changes
AS
  MERGE INTO orders_target t
  USING orders_stream s ON t.order_id = s.order_id
  WHEN MATCHED THEN UPDATE SET t.amount = s.amount
  WHEN NOT MATCHED THEN INSERT VALUES (s.order_id, s.amount);

-- SERVERLESS TASK (auto-managed compute)
CREATE TASK serverless_cleanup
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'MEDIUM'
  SCHEDULE = 'USING CRON 0 2 * * * UTC' -- 2am UTC daily
AS
  DELETE FROM audit_log WHERE log_date < DATEADD('day', -90, CURRENT_DATE());

-- DAG TASKS (pipeline with dependencies)
-- Root: has a schedule
CREATE TASK dag_root
  WAREHOUSE = etl_wh
  SCHEDULE = 'USING CRON 0 1 * * * America/New_York' -- 1am ET daily
AS CALL sp_extract_data();

-- Child 1: runs after root completes
CREATE TASK dag_transform
  WAREHOUSE = etl_wh
  AFTER dag_root -- dependency
AS CALL sp_transform_data();

-- Child 2: runs after transform
CREATE TASK dag_load
  WAREHOUSE = etl_wh
  AFTER dag_transform
AS CALL sp_load_data();
```

EXAM Tasks are ALWAYS created in SUSPENDED state — you MUST run ALTER TASK ... RESUME. In a DAG, resume I

6.3 SCD Type 2 — Full Implementation

Slowly Changing Dimension Type 2 (SCD2) maintains full history by adding new rows for each change rather than overwriting. Each record has effective_start, effective_end, and is_current columns.

SCD Type 2 — Complete Implementation

```
-- Source table (updated by operational system)
CREATE TABLE src_customers (
  customer_id  NUMBER,
  name         VARCHAR,
  email        VARCHAR,
  region       VARCHAR,
  updated_at   TIMESTAMP_NTZ
);

-- SCD2 dimension table
CREATE TABLE dim_customers (
  sk_customer   NUMBER AUTOINCREMENT PRIMARY KEY, -- surrogate key
  customer_id   NUMBER,                          -- natural/business key
  name         VARCHAR,
  email        VARCHAR,
  region       VARCHAR,
  eff_start_date DATE,
  eff_end_date  DATE, -- NULL means current record
  is_current    BOOLEAN DEFAULT TRUE,
  dw_updated_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Stream on source
CREATE STREAM src_customers_stream ON TABLE src_customers;

-- Stored procedure for SCD2 merge
CREATE OR REPLACE PROCEDURE sp_scd2_customers()
RETURNS STRING LANGUAGE SQL AS
$$
BEGIN
  -- Step 1: Expire changed records (set end date on old current row)
  UPDATE dim_customers d
  SET eff_end_date = CURRENT_DATE() - 1,
      is_current   = FALSE,
      dw_updated_at = CURRENT_TIMESTAMP()
  FROM (
    SELECT s.customer_id
    FROM src_customers_stream s
    JOIN dim_customers dc
      ON s.customer_id = dc.customer_id AND dc.is_current = TRUE
    WHERE s.METADATA$ACTION = 'INSERT'
      AND s.METADATA$ISUPDATE = TRUE
      AND (s.name != dc.name OR s.email != dc.email OR s.region != dc.region)
  ) changed
  WHERE d.customer_id = changed.customer_id AND d.is_current = TRUE;

  -- Step 2: Insert new current rows (net-new + changed)
```

6.4 Task Monitoring with AWS SNS

Task Monitoring & SNS Notifications

```
-- Step 1: Create SNS notification integration
CREATE NOTIFICATION INTEGRATION sns_int
  TYPE                = QUEUE
  NOTIFICATION_PROVIDER = AWS_SNS
  ENABLED              = TRUE
  AWS_SNS_TOPIC_ARN    = 'arn:aws:sns:us-east-1:123456789:snowflake-alerts'
  AWS_SNS_ROLE_ARN     = 'arn:aws:iam::123456789:role/SnowflakeNotifyRole';

DESC INTEGRATION sns_int; -- get IAM details to add to SNS trust policy

-- Step 2: Create task with error notification
CREATE TASK monitored_etl_task
  WAREHOUSE      = etl_wh
  SCHEDULE       = '1 HOUR'
  ERROR_INTEGRATION = sns_int -- SNS alert on failure
AS CALL sp_scd2_customers();

ALTER TASK monitored_etl_task RESUME;

-- Step 3: Success notification via child task
CREATE TASK notify_success_task
  WAREHOUSE = etl_wh
  AFTER      monitored_etl_task -- runs only on parent SUCCESS
AS
  CALL SYSTEM$SEND_EMAIL(
    'my_email_integration',
    'team@company.com',
    'ETL Task Succeeded',
    'monitored_etl_task completed successfully at: '
      || CURRENT_TIMESTAMP()::VARCHAR
  );

ALTER TASK notify_success_task RESUME;

-- Step 4: Monitor task history
SELECT NAME, STATE, ERROR_CODE, ERROR_MESSAGE,
  SCHEDULED_TIME, COMPLETED_TIME, RETURN_VALUE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
  SCHEDULED_TIME_RANGE_START =>DATEADD('day', -1, CURRENT_TIMESTAMP())
))
WHERE NAME IN ('MONITORED_ETL_TASK', 'NOTIFY_SUCCESS_TASK')
ORDER BY SCHEDULED_TIME DESC;
```

CHAPTER 7

Security & Governance

Snowflake security is built on role-based access control (RBAC). Every privilege is granted to a role, and roles are assigned to users. Multiple layers of security can be stacked for fine-grained control.

7.1 RBAC — Role-Based Access Control

Snowflake ships with five system roles. Custom roles are created and organized into a hierarchy. Roles inherit privileges from roles granted to them.

System Role	Purpose	Best Practice
ORGADMIN	Manage the Snowflake organization (multiple accounts)	Use rarely — org-level operations only
ACCOUNTADMIN	Full account control — billing, users, objects	NEVER use for daily work — emergency only
SECURITYADMIN	Create and manage users, roles, and security policies	Use for security administration
USERADMIN	Create users and roles only	Use for routine user management
SYSADMIN	Create and manage databases, warehouses, schemas, tables	Daily administration role
PUBLIC	Automatically assigned to every user	Minimal privileges only

RBAC — Full Setup

```
-- Create and organize custom roles
CREATE ROLE analyst_role;
CREATE ROLE senior_analyst_role;
CREATE ROLE data_engineer_role;

-- Role hierarchy: senior_analyst inherits analyst privileges
GRANT ROLE analyst_role TO ROLE senior_analyst_role;
GRANT ROLE senior_analyst_role TO ROLE data_engineer_role;

-- Grant privileges: 4 things needed to query a table
GRANT USAGE ON DATABASE sales_db TO ROLE analyst_role;
GRANT USAGE ON SCHEMA sales_db.public TO ROLE analyst_role;
GRANT SELECT ON ALL TABLES IN SCHEMA sales_db.public TO ROLE analyst_role;
GRANT USAGE ON WAREHOUSE analytics_wh TO ROLE analyst_role;
-- Missing ANY of these 4 = query fails!

-- Future tables automatically get the same privilege
GRANT SELECT ON FUTURE TABLES IN SCHEMA sales_db.public TO ROLE analyst_role;
GRANT SELECT ON FUTURE VIEWS IN SCHEMA sales_db.public TO ROLE analyst_role;

-- Write access for ETL role
GRANT INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA sales_db.public TO ROLE data_engineer_role;

-- Assign role to user
```

7.2 Dynamic Data Masking

Dynamic Data Masking

```
-- Masking policy: controls column value visibility by role
CREATE MASKING POLICY email_mask
AS (val STRING) RETURNS STRING ->
CASE
  WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN', 'DATA_OWNER') THEN val
  WHEN CURRENT_ROLE() = 'ANALYST_ROLE'
    THEN CONCAT(LEFT(val,2), '***@***.com')      -- partial mask
  ELSE '***MASKED***'                          -- full mask
END;

-- Credit card: show last 4 only
CREATE MASKING POLICY cc_mask
AS (val STRING) RETURNS STRING ->
CASE
  WHEN CURRENT_ROLE() = 'FINANCE_ROLE' THEN val
  ELSE CONCAT('****-****-****-', RIGHT(val,4))
END;

-- Numeric masking: round salary for managers
CREATE MASKING POLICY salary_mask
AS (val NUMBER) RETURNS NUMBER ->
CASE
  WHEN CURRENT_ROLE() = 'HR_ROLE' THEN val
  WHEN CURRENT_ROLE() = 'MANAGER_ROLE' THEN ROUND(val, -4)
  ELSE NULL
END;

-- Apply policies to columns
ALTER TABLE customers MODIFY COLUMN email SET MASKING POLICY email_mask;
ALTER TABLE payments MODIFY COLUMN credit_card SET MASKING POLICY cc_mask;
ALTER TABLE employees MODIFY COLUMN salary SET MASKING POLICY salary_mask;

-- Unset a policy
ALTER TABLE customers MODIFY COLUMN email UNSET MASKING POLICY;

-- Inspect policies
SHOW MASKING POLICIES;
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.POLICY_REFERENCES
WHERE POLICY_NAME = 'EMAIL_MASK';
```

7.3 Row Access Policies

Row Access Policies

```
-- Row access policy: controls WHICH ROWS are visible
-- Mapping table: username → allowed regions
CREATE TABLE region_access_map (
  username VARCHAR,
  region VARCHAR
);

INSERT INTO region_access_map VALUES
  ('ALICE', 'US'), ('BOB', 'EMEA'), ('BOB', 'APAC');
```

```
-- Policy: each user sees only their mapped regions
```

7.4 Secure Data Sharing

Data Sharing

```
-- PROVIDER SIDE: share live data with another account
CREATE SHARE sales_share;

GRANT USAGE ON DATABASE sales_db TO SHARE sales_share;
GRANT USAGE ON SCHEMA sales_db.public TO SHARE sales_share;

-- Share via secure view (recommended – add filtering/masking in view)
CREATE SECURE VIEW sales_db.public.orders_shared AS
  SELECT order_id, product_category, region, amount, order_date
  FROM orders
  WHERE partner_visible = TRUE;

GRANT SELECT ON VIEW sales_db.public.orders_shared TO SHARE sales_share;
ALTER SHARE sales_share ADD ACCOUNTS = consumer_account_id;

-- CONSUMER SIDE: create database from share (no data copied)
CREATE DATABASE shared_sales FROM SHARE provider_acct.sales_share;
SELECT * FROM shared_sales.public.orders_shared;

-- READER ACCOUNT (for non-Snowflake consumers)
CREATE MANAGED ACCOUNT reader_partner
  ADMIN_NAME = 'partner_admin'
  ADMIN_PASSWORD = 'Str0ngP@ss123!'
  TYPE = READER;
ALTER SHARE sales_share ADD ACCOUNTS = reader_partner;

-- Network policy to restrict reader account by IP
CREATE NETWORK POLICY partner_ip_policy
  ALLOWED_IP_LIST = ('203.0.113.0/24');
ALTER ACCOUNT reader_partner SET NETWORK_POLICY = partner_ip_policy;
```

CHAPTER 8

Advanced Features

Stored procedures, UDFs, Snowpark, resource monitors, and dynamic tables extend Snowflake from a SQL engine to a full data platform.

8.1 Stored Procedures

Language	Best For	Key Feature
SQL Scripting	Simple DML, IF/ELSE, loops, variable assignment	Native, no framework needed
JavaScript	Complex control flow, dynamic SQL, error handling	Full JS runtime, cursors
Python	Pandas, ML libraries, data science logic	Rich ecosystem, DataFrames
Java/Scala	Snowpark enterprise pipelines	JVM ecosystem

Stored Procedures — All Types

```
-- SQL SCRIPTING (simplest – native Snowflake Scripting)
CREATE OR REPLACE PROCEDURE sp_cleanup(days_old NUMBER)
RETURNS STRING LANGUAGE SQL AS
$$
DECLARE
  deleted_count NUMBER DEFAULT 0;
  start_date    DATE DEFAULT DATEADD('day', -:days_old, CURRENT_DATE());
BEGIN
  DELETE FROM orders WHERE order_date < start_date;
  deleted_count := SQLROWCOUNT;

  IF (deleted_count > 0) THEN
    INSERT INTO audit_log VALUES (CURRENT_TIMESTAMP(), 'cleanup', deleted_count);
  END IF;

  RETURN 'Deleted ' || deleted_count || ' rows before ' || start_date;
EXCEPTION
  WHEN OTHER THEN
    RETURN 'Error: ' || SQLERRM;
END;
$$;
CALL sp_cleanup(90);

-- JAVASCRIPT (complex control flow, error handling)
CREATE OR REPLACE PROCEDURE sp_dynamic_table_stats(db_name STRING)
RETURNS VARIANT LANGUAGE JAVASCRIPT AS
$$
  var stats = [];
  var tables = snowflake.execute({
    sqlText: `SHOW TABLES IN DATABASE ${DB_NAME}`
  });
  while (tables.next()) {
    var tname = tables.getColumnValue('name');
    var stats = snowflake.execute({sqlText: `SELECT COUNT(*) AS c FROM ${DB_NAME}.${tname}`});
  }
  return {db_name: db_name, stats: stats.getColumnValue('c')};
$;
```

8.2 UDFs & UDTFs

UDFs & UDTFs

```
-- SCALAR UDF – one return value per row
CREATE OR REPLACE FUNCTION classify_order(amount FLOAT)
RETURNS VARCHAR
LANGUAGE SQL AS $$
    CASE
        WHEN amount < 100 THEN 'small'
        WHEN amount < 1000 THEN 'medium'
        ELSE 'large'
    END
$$;

SELECT order_id, classify_order(amount) AS order_size FROM orders;

-- PYTHON VECTORIZED UDF (batch processing – much faster than row-by-row)
CREATE OR REPLACE FUNCTION clean_phone(phone STRING)
RETURNS STRING
LANGUAGE PYTHON RUNTIME_VERSION='3.10'
PACKAGES=('pandas')
HANDLER='compute'
AS $$
import pandas as pd, re

def compute(phone_series: pd.Series) -> pd.Series:
    return phone_series.str.replace(r'^0-9', '', regex=True)
$$;

-- UDTF – User-Defined TABLE Function (returns multiple rows per input)
CREATE OR REPLACE FUNCTION split_tags(tag_str STRING)
RETURNS TABLE (tag VARCHAR)
LANGUAGE SQL AS $$
    SELECT TRIM(VALUE)
    FROM TABLE(SPLIT_TO_TABLE(tag_str, ','))
$$;

-- Use UDTF in a query
SELECT p.product_id, t.tag
FROM products p,
TABLE(split_tags(p.tags)) t;

-- Show / drop UDFs
SHOW USER FUNCTIONS;
DROP FUNCTION classify_order(FLOAT);
```

8.3 Snowpark

Snowpark Python

```
# Snowpark Python – push computation to Snowflake
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, sum as sum_, udf
from snowflake.snowpark.types import StringType, FloatType
```

```
# Connect
session = Session.builder.configs({
    'snowflake.url': 'https://snowflake.us-east-1.amazonaws.com',
    'snowflake.username': 'snowflake',
    'snowflake.password': 'snowflake'
}).create()
```

8.4 Resource Monitors

Resource Monitors

```
-- Create resource monitor with escalating triggers
CREATE RESOURCE MONITOR prod_monthly_cap
  CREDIT_QUOTA    = 2000          -- 2000 credits per month
  FREQUENCY       = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 50 PERCENT DO NOTIFY          -- email alert
    ON 75 PERCENT DO NOTIFY
    ON 90 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND        -- suspend after current queries finish
    ON 110 PERCENT DO SUSPEND_IMMEDIATE; -- kill running queries immediately

-- Apply to entire account
ALTER ACCOUNT SET RESOURCE_MONITOR = prod_monthly_cap;

-- Apply per warehouse (per-team cost control)
ALTER WAREHOUSE analytics_wh SET RESOURCE_MONITOR = prod_monthly_cap;
ALTER WAREHOUSE etl_wh      SET RESOURCE_MONITOR = etl_monthly_cap;

-- Check current usage
SHOW RESOURCE MONITORS;

-- Credit usage tracking
SELECT USAGE_DATE, CREDITS_USED_COMPUTE, CREDITS_USED_CLOUD_SERVICES,
       CREDITS_USED_COMPUTE + CREDITS_USED_CLOUD_SERVICES AS total
FROM   SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE  USAGE_DATE >= DATE_TRUNC('month', CURRENT_DATE())
ORDER BY USAGE_DATE;
```

CHAPTER 9

Interview Questions — Technical & Scenario-Based

50 technical questions covering all Snowflake topics, followed by 40 scenario-based questions that test real-world problem solving. All questions are calibrated for 5+ years experience.

9.1 Technical Interview Questions

Architecture

[Easy] What are the three layers of Snowflake architecture?

Cloud Services (auth, optimization, metadata), Query Processing (virtual warehouses), Storage (compressed columnar micro-partitions in S3/Azure/GCS). Key insight: storage and compute are completely separated.

[Easy] What is a virtual warehouse and how is it billed?

A compute cluster that executes queries. Billed per second with a 60-second minimum on each resume. Pausing stops billing immediately. Cost doubles with each size step up.

[Medium] What is the difference between scaling up and scaling out?

Scale UP: bigger warehouse size (XS→XL) — more CPU/RAM per query. Helps slow, complex queries.
Scale OUT: multi-cluster warehouse — adds clusters for concurrent users. Helps query queue depth.
Enterprise edition required for multi-cluster.

[Hard] How does Snowflake handle concurrent queries from 500 users?

Multi-cluster warehouse auto-scales to add compute clusters when queue depth grows. Each cluster handles an independent pool of queries. Scaling policy: STANDARD adds clusters quickly, ECONOMY waits longer to save cost. Single-cluster would queue queries; multi-cluster scales horizontally.

[Hard] What is Snowflake's transaction isolation level and what does it mean in practice?

Snapshot Isolation (SI). Each transaction sees a consistent snapshot of data at its start time. Prevents dirty reads, non-repeatable reads, phantom reads. Concurrent DML can cause different COUNT(*) results between two sequential queries if a DELETE ran between them — both are 'correct' for their snapshot.

Storage & Tables

[Easy] What is the difference between a transient and temporary table?

Transient: persists across sessions, Time Travel up to 1 day, NO Fail-safe, lower storage cost. Temporary: only exists for the current session, auto-dropped when session ends. Both save money vs permanent tables by avoiding 7-day Fail-safe.

[Medium] What is zero-copy cloning and when would you use it?

Instantly copies metadata only — source and clone share micro-partitions until modified. Use cases: dev/test environments from production, pre-migration snapshots, A/B testing transformations. Cost is near-zero initially, grows only as clone diverges from source.

[Hard] How does Time Travel interact with Fail-safe for GDPR right-to-erasure requests?

Deleting data leaves it in Time Travel (up to 90 days) and then Fail-safe (7 more days). True erasure requires: set `DATA_RETENTION_TIME=0` BEFORE deleting, then delete. Existing Time Travel data requires waiting out retention or contacting Snowflake Support. Best practice: tokenize/pseudonymize PII at ingestion so token revocation renders data meaningless.

Data Loading

[Easy] What is the difference between an internal stage and an external stage?

Internal: managed by Snowflake in its own cloud storage (user, table, or named). External: points to your S3/Azure/GCS bucket — files stay in your storage. Choose external when files are already in your cloud, need to share with other systems, or compliance requires you own the data.

[Medium] How does Snowflake prevent duplicate loads in COPY INTO?

Snowflake tracks file ETags in a load history per stage+table for 64 days. Re-running COPY INTO for already-loaded files skips them automatically. Use `FORCE=TRUE` to override (risks duplicates). After 64 days, history expires and files can be re-loaded.

[Medium] What is Snowpipe and how does AUTO_INGEST work?

Serverless continuous loading triggered by events. `AUTO_INGEST=TRUE`: Snowflake provides an SQS ARN. Configure S3 event notification (ObjectCreated) to send to that SQS. New file in S3 → event → SQS → Snowpipe runs COPY INTO automatically. Latency: 1–5 minutes.

[Hard] A 10TB migration from S3 has 50,000 files. How do you load it as fast as possible?

Use LARGE or XL warehouse (more parallel threads). Run multiple COPY INTO sessions in parallel with different PATTERN filters (e.g., session 1: 2022 files, session 2: 2023 files). COPY INTO parallelizes across files — more files = better parallelism. Files should be 100-250MB compressed.

Performance

[Easy] What three things must be true for the result cache to serve a query?

1) Exact same SQL text (even whitespace matters). 2) Underlying data has not changed since last cache. 3) Query uses no non-deterministic functions (`CURRENT_TIMESTAMP`, `RANDOM`, `UUID_STRING`).

[Medium] What does 'Bytes spilled to remote storage' in Query Profile mean?

The warehouse ran out of SSD space during a sort/join/aggregation and spilled to S3/Azure/GCS. This is the slowest possible execution path. Fix: size up the warehouse to get more total SSD per query.

[Hard] You added a clustering key on order_date but query performance did not improve. Why?

Likely causes: 1) Reclustering is still in progress — check `SYSTEM$CLUSTERING_INFORMATION`. 2) Queries filter on columns OTHER than `order_date` — clustering only helps the specified key columns. 3) Table is too small to benefit — clustering overhead outweighs gains on small tables. 4) Result cache is serving queries — performance was already good.

Security

[Easy] What four grants does a user need to run a `SELECT` query on a table?

1) `GRANT USAGE ON DATABASE`. 2) `GRANT USAGE ON SCHEMA`. 3) `GRANT SELECT ON TABLE`. 4) `GRANT USAGE ON WAREHOUSE`. Missing any one = query fails with insufficient privilege error.

[Medium] What is the difference between a masking policy and a row access policy?

Masking policy: applies to a `COLUMN` — hides or transforms the value (row still returned). Row access policy: applies to a `TABLE` — filters which `ROWS` are visible at all. They can be combined: user sees only their region's rows (row policy) with masked credit cards (masking policy).

[Hard] `EXECUTE AS OWNER` vs `EXECUTE AS CALLER` in stored procedures — when would you use each?

`EXECUTE AS CALLER` (default): procedure runs with the calling user's role privileges — safer, limited blast radius. Use for user-facing procedures where you want the caller's access enforced. `EXECUTE AS OWNER`: runs with the procedure owner's (usually `SYSADMIN`) privileges — use for ETL procedures that need elevated access regardless of who calls them. Carefully audit what `OWNER` procedures do.

Pipelines

[Easy] Tasks are created in what state and what command activates them?

Tasks are created in `SUSPENDED` state. Run `ALTER TASK task_name RESUME` to activate. In a DAG, always resume leaf (child) tasks first and root task last.

[Medium] What is the difference between a standard stream and an append-only stream?

Standard: tracks `INSERT`, `UPDATE`, `DELETE`. Append-only: tracks `INSERT` only — no `DELETE` or `UPDATE` rows in the stream. More efficient for log/event tables where data is only ever added, never modified.

[Hard] Explain exactly-once semantics with Snowflake streams.

Stream offset advances ONLY when a DML statement that consumes the stream commits successfully. If the task fails or is aborted, the offset does not advance — same changes are reprocessed on retry. To achieve exactly-once: use `MERGE` (not `INSERT`) so reprocessing the same rows is idempotent. `MERGE` with `event_id` as join key: `duplicate event_id = WHEN MATCHED DO NOTHING`.

9.2 Scenario-Based Interview Questions

The following scenarios mirror real production problems. At 5+ years, interviewers expect a structured approach: diagnose → fix → prevent recurrence.

Performance Scenarios

[Hard] Diagnose and fix the performance degradation.

Scenario: Your analytics dashboard queries grew from 5 seconds to 5 minutes after a table scaled from 50GB to 2TB.

Approach:

1) Open Query Profile → check Partitions scanned/total. High ratio = pruning problem. 2) Run `SYSTEM$CLUSTERING_INFORMATION` — check `average_depth`. 3) Identify filter columns in all dashboard queries (likely date). 4) Add clustering key: `ALTER TABLE fact_sales CLUSTER BY (order_date)`. 5) Monitor reclustering progress with `SYSTEM$CLUSTERING_INFORMATION`.

SQL Implementation

```
-- Check clustering health
SELECT SYSTEM$CLUSTERING_INFORMATION('fact_sales', '(order_date)');
-- average_depth > 3 means poor clustering

-- Add clustering key
ALTER TABLE fact_sales CLUSTER BY (order_date);

-- Monitor partition scan ratio after
EXPLAIN SELECT * FROM fact_sales WHERE order_date = '2024-03-15';
-- partitionsAssigned / partitionsTotal should drop significantly
```

EXAM Key Insight: New data inserted over months mixes partition date ranges, making pruning ineffective. Clustering reorg

[Medium] Is this a bug? What is happening?

Scenario: The same query returns in 0ms sometimes and 30 seconds other times. Data hasn't changed.

Approach:

Not a bug — this is the result cache working correctly. 0ms = result cache hit (cached within 24h, data unchanged, no non-deterministic functions). 30s = cache miss (after 24h expiry, or data changed, or non-deterministic function in query). If `CURRENT_TIMESTAMP` or `RANDOM()` is in the query, it always bypasses cache.

SQL Implementation

```
-- Test result cache behavior
ALTER SESSION SET USE_CACHED_RESULT = FALSE; -- force miss for testing
ALTER SESSION SET USE_CACHED_RESULT = TRUE;  -- re-enable

-- These ALWAYS miss result cache:
SELECT COUNT(*), CURRENT_TIMESTAMP() FROM orders;
SELECT COUNT(*), RANDOM() FROM orders;

-- Check Query History: 0ms execution = result cache hit
```

EXAM Key Insight: Result cache lasts 24h from last hit. Non-deterministic functions always bypass it by design.

[Hard] Why didn't sizing up help and what is the correct solution?

Scenario: 200 concurrent BI users are queuing on an XL warehouse. Sizing to 2XL did not help.

Approach:

Scale UP adds more CPU/RAM per query — helps slow queries, not concurrency. 200 users queueing = concurrency problem. Solution: enable multi-cluster warehouse. Set MAX_CLUSTER_COUNT=5 to auto-scale to 5 clusters. Each cluster handles separate query pools. Requires Enterprise edition.

SQL Implementation

```
ALTER WAREHOUSE bi_wh
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 5
  SCALING_POLICY     = 'STANDARD'; -- ECONOMY waits longer, saves cost

-- Monitor cluster utilization
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_LOAD_HISTORY
WHERE WAREHOUSE_NAME = 'BI_WH'
  AND START_TIME > DATEADD('hour', -6, CURRENT_TIMESTAMP());
```

EXAM Key Insight: Scale UP = faster individual queries. Scale OUT = more concurrent queries. Classic interview distinction.

Data Loading Scenarios

[Hard] How do you diagnose and fix Snowpipe failure?

Scenario: Snowpipe was working for 3 months. Today files land in S3 but nothing loads. No errors visible.

Approach:

1) SYSTEM\$PIPE_STATUS — check executionState, pendingFileCount, notificationChannelStatus. 2) SHOW PIPES — check if pipe is paused. 3) Check AWS: was the S3 event notification modified? Was the SQS queue deleted/modified? 4) Check IAM role — were permissions on the S3 bucket changed? 5) ALTER PIPE REFRESH to catch up on missed files.

SQL Implementation

```
-- Step 1: Check pipe status
SELECT SYSTEM$PIPE_STATUS('orders_pipe');

-- Step 2: Resume if paused
ALTER PIPE orders_pipe SET PIPE_EXECUTION_PAUSED = FALSE;

-- Step 3: Catch up on missed files
ALTER PIPE orders_pipe REFRESH
  MODIFIED_AFTER = '2024-03-20T00:00:00';

-- Step 4: Check error history
SELECT * FROM TABLE(INFORMATION_SCHEMA.PIPE_USAGE_HISTORY(
  DATE_RANGE_START => DATEADD('day', -1, CURRENT_TIMESTAMP()),
  PIPE_NAME => 'ORDERS_PIPE'
));
```

EXAM Key Insight: Most common root causes: SQS queue ARN changed (S3 notification misconfigured), IAM role permissions

[Medium] How do you identify and remove the duplicates?

Scenario: A junior engineer ran COPY INTO with FORCE=TRUE and duplicated 3 days of data.

Approach:

Option 1 (fastest): Time Travel — find the QUERY_ID of the bad COPY via QUERY_HISTORY, clone the table BEFORE that statement, swap tables. Option 2: Deduplicate with QUALIFY ROW_NUMBER — create clean table, drop original, rename.

SQL Implementation

```
-- Option 1: Time Travel recovery
SELECT QUERY_ID, START_TIME FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TEXT ILIKE '%COPY INTO orders%FORCE%'
ORDER BY START_TIME DESC LIMIT 5;

CREATE TABLE orders_clean CLONE orders
  BEFORE (STATEMENT => '<bad_copy_query_id>');
ALTER TABLE orders RENAME TO orders_duped_backup;
ALTER TABLE orders_clean RENAME TO orders;

-- Option 2: QUALIFY dedup
CREATE TABLE orders_deduped AS
SELECT * FROM orders
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id ORDER BY loaded_at DESC
) = 1;
```

EXAM Key Insight: Time Travel is the fastest recovery within retention window. QUALIFY is the cleanest SQL dedup pattern

Pipeline Scenarios

[Hard] What caused this and how do you fix it?

Scenario: Your SCD2 pipeline has duplicate is_current=TRUE rows for some customers after 6 months of operation.

Approach:

Root cause: race condition — two task executions ran concurrently (or nearly so), both ran the INSERT step without seeing the other's row, creating two current records. Fix: add QUALIFY to deduplicate stream before MERGE, make MERGE idempotent, ensure only one task runs at a time.

SQL Implementation

```
-- Step 1: Find duplicates
SELECT customer_id, COUNT(*) AS cnt
FROM dim_customers WHERE is_current = TRUE
GROUP BY 1 HAVING cnt > 1;

-- Step 2: Remove duplicates (keep latest)
```

EXAM Key Insight: Snowflake UNIQUE constraints are informational — they don't enforce. Always QUALIFY the stream source.

[Hard] What happens downstream and how do you recover?

Scenario: A task DAG has 5 steps: `extract`→`validate`→`transform`→`load`→`notify`. 'transform' fails.

Approach:

Failed parent → all downstream tasks get SKIPPED state (not FAILED). Stream offset is NOT advanced (DML didn't commit). Stream data is preserved — safe to reprocess. Fix the transform logic, then EXECUTE TASK on the root or transform task.

SQL Implementation

```
-- Check task states
SELECT NAME, STATE, ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('hour', -2, CURRENT_TIMESTAMP())
))
ORDER BY SCHEDULED_TIME DESC;
-- transform: FAILED, load: SKIPPED, notify: SKIPPED

-- Fix the transform stored procedure, then re-run from root
EXECUTE TASK extract_task;

-- Or re-run from the failed step only
EXECUTE TASK transform_task;
```

EXAM Key Insight: Failed parent = downstream SKIPPED. Stream data preserved. EXECUTE TASK re-runs from any point.

Security Scenarios

[Hard] How do you handle right-to-erasure given Time Travel and Fail-safe?

Scenario: GDPR request: delete all data for `customer_id=12345` including historical SCD2 records.

Approach:

1) Set `DATA_RETENTION_TIME=0` on affected tables BEFORE deletion (prevents Time Travel capturing new deletes). 2) Delete from all tables (current + SCD2 history + fact tables). 3) For existing Time Travel: wait out retention, or contact Snowflake Support for Fail-safe. 4) Best practice: tokenize PII at ingestion — revoking token renders all data meaningless instantly.

SQL Implementation

```
-- Set retention to 0 BEFORE deleting
ALTER TABLE dim_customers SET DATA_RETENTION_TIME_IN_DAYS = 0;
ALTER TABLE fact_orders SET DATA_RETENTION_TIME_IN_DAYS = 0;

-- Delete from all tables
DELETE FROM dim_customers WHERE natural_key = 12345;
DELETE FROM fact_orders WHERE customer_id = 12345;
DELETE FROM customers WHERE customer_id = 12345;
```

```
-- Verify deletion
```

EXAM Key Insight: Setting `DATA_RETENTION_TIME=0` BEFORE deletion prevents Time Travel from capturing the delete.

[Medium] Implement column-level security without changing every query.

Scenario: Security audit finds analysts can read salary data they shouldn't. 500 analysts, 10 salary columns.

Approach:

Create a masking policy that returns NULL (or masked value) for any role other than `HR_ROLE`. Apply to each salary column. Policy is transparent — analysts write normal SQL, masking applies automatically.

SQL Implementation

```
CREATE MASKING POLICY salary_mask
AS (val NUMBER) RETURNS NUMBER ->
CASE
  WHEN CURRENT_ROLE() IN ('HR_ROLE', 'ACCOUNTADMIN') THEN val
  WHEN CURRENT_ROLE() = 'MANAGER_ROLE' THEN ROUND(val, -4)
  ELSE NULL
END;

-- Apply to column
ALTER TABLE employees
  MODIFY COLUMN salary SET MASKING POLICY salary_mask;

-- Test
USE ROLE analyst_role;
SELECT name, salary FROM employees LIMIT 3;
-- salary = NULL for analysts
```

EXAM Key Insight: Masking is applied per role at query time. 500 analysts need no individual changes — the policy covers all.

Cost Scenarios

[Hard] Walk through your full cost investigation process.

Scenario: Snowflake bill doubled last month with no new projects or users added.

Approach:

1) `METERING_DAILY_HISTORY` — find the spike day. 2) `WAREHOUSE_METERING_HISTORY` — find which warehouse drove it. 3) `QUERY_HISTORY` on that warehouse on spike days — find top consumers. 4) Check `AUTO_SUSPEND` — was it disabled? 5) `TABLE_STORAGE_METRICS` — check Time Travel and Fail-safe growth.

SQL Implementation

```
-- Find spike day
SELECT USAGE_DATE, CREDITS_USED_COMPUTE
FROM SNOWFLAKE.ACCOUNT_USAGE.METERING_DAILY_HISTORY
WHERE USAGE_DATE >= DATEADD('month', -2, CURRENT_DATE())
ORDER BY USAGE_DATE;
```

```
-- Find culprit warehouse
```

EXAM Key Insight: Always start with METERING_DAILY_HISTORY to pinpoint the day, then drill down. Most surprise bills:

CHAPTER 10

Quick Reference — Exam Cheat Sheet

The most important facts distilled. These are the answers that separate 5-year engineers from 2-year engineers.

Virtual Warehouses

- Billing: per-second, 60-second MINIMUM per resume
- AUTO_SUSPEND=60 is the golden default for most warehouses
- Suspending clears local disk cache (SSD)
- Scale UP for slow queries → Scale OUT (multi-cluster) for concurrent users
- Snowpark-optimized = 16x memory per node (for ML/Python workloads)
- Multi-cluster requires Enterprise edition

Caching

- Result cache: 24h, Cloud Services layer, NO warehouse needed
- CURRENT_TIMESTAMP / RANDOM / UUID_STRING always bypass result cache
- Local disk cache: warehouse SSD, cleared on suspend/resume
- Metadata cache: always available, serves COUNT(*) / MIN / MAX instantly

Tables

- Permanent: Time Travel up to 90d + 7-day Fail-safe (Enterprise)
- Transient: Time Travel up to 1d, NO Fail-safe — save storage cost on staging tables
- Temporary: session-scoped, auto-dropped, no Time Travel after session
- External: no Time Travel, no Fail-safe, no clustering — files stay in your cloud
- Dynamic: declarative incremental — specify TARGET_LAG, Snowflake manages refresh

Data Loading

- All data must pass through a STAGE before COPY INTO
- COPY INTO tracks load history for 64 days — same files won't double-load
- FORCE=TRUE overrides load history — risks duplicates
- Snowpipe AUTO_INGEST: SQS queue ARN from SHOW PIPES → set S3 event notification
- Optimal file size: 100–250 MB compressed
- VALIDATION_MODE='RETURN_ERRORS' = dry run without loading

RBAC — The Four Required Grants

- 1. GRANT USAGE ON DATABASE
- 2. GRANT USAGE ON SCHEMA
- 3. GRANT SELECT ON TABLE
- 4. GRANT USAGE ON WAREHOUSE
- Missing ANY ONE of these four = query fails!
- GRANT SELECT ON FUTURE TABLES IN SCHEMA = auto-grant for new tables

Streams & Tasks

- Stream offset advances ONLY on successful DML commit — retries are safe
- Plain SELECT on stream does NOT advance offset
- METADATA\$ACTION = 'INSERT' or 'DELETE'; updates appear as DELETE+INSERT pair
- Tasks are created SUSPENDED — always ALTER TASK ... RESUME
- DAG: resume LEAF tasks first, ROOT task last
- EXECUTE TASK = manual run bypassing schedule (for testing)
- WHEN SYSTEM\$STREAM_HAS_DATA = skip task run if nothing changed

Security

- Masking policy: hides column VALUE, row still returned
- Row access policy: hides entire ROW — user doesn't know it exists
- Secure view: hides SQL definition — required for Data Sharing
- EXECUTE AS OWNER: runs with procedure owner's privileges
- EXECUTE AS CALLER: runs with calling user's privileges (safer default)
- Data Sharing: live data, no copy, consumer pays for compute

Performance

- Partition pruning: WHERE on clustered column skips non-matching partitions
- Clustering key: add only for tables >1TB with consistent filter patterns
- Spill to local/remote storage: warehouse too small → size up
- SELECT * kills columnar benefit → always project only needed columns
- Join key type mismatch → implicit cast → full scan → always match types
- QUALIFY = filter on window functions (no subquery needed)

Cost Management

- Cloud services > 10% of compute = billed separately
- Common cause: SHOW commands in loops by BI tools

- Transient tables save ~30% storage (no Fail-safe overhead)
- Resource monitors: NOTIFY → SUSPEND → SUSPEND_IMMEDIATE
- SUSPEND_IMMEDIATE kills running queries at the credit threshold
- Use serverless tasks for variable-volume pipelines (pay per use)

Snowflake Complete Reference Guide

Covers all topics for SnowPro Core Certification and 5+ Year Experience Interviews

Topics: Architecture · Tables · Data Loading · SQL & Performance · Views · Streams & Tasks · SCD2 · Security & Governance · Stored Procedures · UDFs · Snowpark · Resource Monitors · Cost Management · Technical Interview Questions · Scenario-Based Interview Questions